

Assignment 1: Data Collection and Preprocessing for Foundation Model Pre-Training

1.42 GB cleaned text | 487,911 documents | 335.3M GPT-2 tokens | 3 domains | generated from pipeline manifests

1. Dataset sources and total size

The corpus is drawn from three public Hugging Face datasets chosen to span the three domains the assignment names: encyclopedic, news, and general web text. The byte budget is split evenly between them, and every source is pinned to a dataset repository commit so a re-run fetches byte-identical data.

Source	Domain	Hugging Face id	License	Docs	GB raw
wikipedia	encyclopedic	wikimedia/wikipedia	CC-BY-SA-3.0 / GFDL	189,336	0.50
cc_news	news	vblagoje/cc_news	unknown (CC-News derivative)	212,198	0.50
fineweb	general_web	HuggingFaceFW/fineweb	ODC-BY-1.0	160,536	0.50
TOTAL				562,070	1.50

Table 1. Raw collection. Collected in 366s with zero stream errors; 633 MB on disk after gzip.

The raw budget is 1.5 GB, roughly 1.5x the 1 GB requirement, because the requirement applies *after* cleaning and deduplication necessarily shrinks the corpus. Rather than downloading whole datasets, Stage 1 streams them and stops at the byte budget.

Sampling is stratified by shard. Each dataset is stored as parquet shards that are ordered non-randomly (Wikipedia's are alphabetical), so reading sequentially would have returned only articles beginning with 'A'. Stage 1 instead walks the shards in a seeded random order and takes an equal byte quota from each, visiting 41/41 wikipedia, 5/5 cc_news, 15/15 fineweb shards. Verified on the output, sampled Wikipedia titles span the full alphabet plus accented and CJK initials.

2. Cleaning strategies and reasoning

Stage 2 applies a filter chain ordered cheapest-first, so a document a hash lookup can reject never pays for normalization. Each rule reports what it removed, which is what makes the surviving corpus size predictable rather than discovered at the end.

Filter	Rule	Docs removed	% of input
dup_raw	byte-identical to a document already kept	29,682	5.28%
dup_normalized	identical after normalization (case, markup, whitespace)	83	0.01%
empty	nothing left after markup stripping	0	0.00%
short	fewer than 50 words	44,172	7.86%
word_length	mean word length outside 2 to 15 characters	0	0.00%
symbol_ratio	word characters below 60% of the text	90	0.02%
repetitive	one token exceeds 30% of all tokens	132	0.02%
KEPT		487,911	86.81%

Table 2. Cleaning outcomes. 562,070 documents in, 74,159 removed, 487,911 kept (1.42 GB) in 463s.

Normalization. NFKC Unicode folding, then removal of script and style blocks with their contents, HTML tags and comments, markdown syntax, URLs and reference markers such as [12] and [citation needed], then lowercasing, then control and zero-width character removal, then whitespace collapsing. Order matters: markup is stripped before whitespace is collapsed because removing a tag leaves a gap, and lowercasing follows NFKC so the ligature and full-width folds NFKC performs are themselves lowercased.

Deduplication runs twice. The first pass uses the SHA-256 digest Stage 1 already stored per document, so exact raw duplicates cost a set lookup and nothing else. The second pass hashes the normalized text, catching documents that differ only in casing, markup or whitespace and are duplicates as far as a language model is concerned. Keys are stored as 64-bit

digest prefixes rather than full hex strings: across 562,070 documents the birthday collision probability is about $1.7e-08$, and it keeps the two membership sets near 40 MB instead of 130 MB.

Limitation. Only exact duplicates are removed. Near-duplicate detection via MinHash or SimHash would catch boilerplate-heavy web pages that differ by a timestamp, but it needs an extra dependency and a second pass over the corpus. For a corpus at this scale the exact passes were judged sufficient; at 100 GB they would not be.

3. Tokenization choices

Tokenizer	<code>gpt2 (GPT2Tokenizer, fast=True)</code>
Algorithm	Byte-level BPE, GPT-style
Vocabulary	50,257 tokens
Block size	1,024 tokens
Storage dtype	uint16 (vocab fits in 16 bits)
Separator	<code>< endoftext ></code> (id 50256) appended per document
Total tokens	335,276,032 (325,858 train blocks)
Train / val split	5 per 1000 documents, by stable hash of document id

Table 3. Tokenization configuration. Encoded in 329s.

Byte-level BPE was chosen because the assignment asks for a GPT-style tokenizer and because it cannot emit unknown tokens: every byte sequence is representable, so the multilingual fragments that survive from Wikipedia and general web text still encode cleanly. Vocabulary and block size are GPT-2's own, which keeps the artifact directly usable for pretraining a GPT-2 class model without re-tokenizing.

Packing, not padding. Documents are concatenated with an end-of-text separator and cut into fixed blocks. This is what makes the assignment's requirement to handle sequences longer than the block size fall out for free: a long document spans several blocks instead of being truncated. In this corpus 81,598 documents (16.7%) exceed one block, so truncation would have discarded a substantial share of the text collected. Document lengths are heavily skewed: median 417 tokens, mean 687, p99 4,683, maximum 124,875.

Storage. Token ids are written as a flat uint16 stream. GPT-2's 50,257-entry vocabulary fits in 16 bits, so numpy's default int64 would have quadrupled a multi-gigabyte artifact for no benefit. Block i is simply the token range $[i*B, (i+1)*B)$, which is what allows the loader to address blocks arithmetically instead of maintaining an index.

The validation split is per-document and hash-based, not a tail slice. The corpus is written source by source, so holding out the last 0.5% of tokens would have produced a validation set made entirely of general-web text. Hashing the document id keeps every domain represented and makes the split reproducible without storing an index. Python's built-in hash is salted per process and would have given a different split on every run, so SHA-256 is used instead.

4. Data loader implementation

Stage 4 provides three dataset classes over the token files, covering the two access patterns pretraining actually needs plus the padded variant the assignment asks to see.

Class	Kind	Purpose
<code>PackedBlockDataset</code>	map-style	Memory-mapped random access. Pairs with <code>DataLoader(shuffle=True)</code> ; shuffling is an index permutation.
<code>StreamingBlockDataset</code>	IterableDataset	For corpora larger than RAM. Blocks split across workers by stride; bounded shuffle buffer.
<code>DocumentDataset</code>	map-style	Variable-length documents with a padding collate, to quantify what packing saves.

Table 4. Dataset classes in `src/data_loader.py`.

Memory. Token files are read with numpy memmap, so the corpus costs a page-cache mapping rather than resident memory and a batch materialises only the rows it touches. The memmap is opened lazily on first access, not in the constructor: a memmap created in the parent process and inherited through fork shares a file offset and does not survive pickling to spawned workers, so lazy opening is what makes `num_workers` above zero safe here.

Targets. Each item reads `block_size + 1` tokens and returns `input_ids = t[:-1]` with `labels = t[1:]`, so the loader emits causal-LM training pairs directly and the training step needs no shifting. Packed blocks carry no attention mask because every position is a real token; the mask appears only on the padded path, where it carries information.

Batching, measured. With batch size 8 and block size 1024, the packed loader yields (8, 1024) batches across 325,857 blocks (40,732 batches per epoch) with every position a real token. The same batch size over un-packed documents wastes 44.4% of positions on padding, averaged over 64 batches. Padding also makes the batch shape depend on the longest document in it, so step time varies; packed batches are a constant shape, which is what lets a real training loop hold a fixed memory budget. In the padded path, label positions are set to -100 so that `cross_entropy` ignores them and padding never contributes a gradient.

5. Challenges encountered

5.1 A streaming shuffle that exhausted memory

The first working version of Stage 1 called `IterableDataset.shuffle(buffer_size=10000)` to avoid the alphabetical clustering described in section 1. On an 8 GB machine with no swap this was killed by the OOM killer every time, after crawling at roughly 1.85 kB/s and emitting a steady stream of SSL and 'peer closed connection' errors that made the problem look like a flaky network.

It was not the network: the same host pulled the same parquet files at 27 MB/s with curl, and plain sequential streaming ran at 41 MB/s. Inspecting the iterator chain the library actually builds showed two compounding faults.

```
BufferShuffledExamplesIterable(buffer_size=10000)
-> CyclingMultiSourcesExamplesIterable [10 shards, one thread each]
-> RebatchedArrowExamplesIterable(batch_size=1)
```

First, on the Arrow fast path the shuffle buffer holds 10,000 Arrow tables rather than 10,000 rows, and those tables are zero-copy slices, so each one pins its entire parent parquet row group in memory. Second, the cycling iterable opens ten shards concurrently, which is what produced the dropped connections: ten simultaneous long-lived range requests against one repository.

The fix replaced the library shuffle with the stratified shard walk described in section 1. One shard is open at a time, so one connection is open at a time, and resident memory stays flat at roughly 670 MB. The full 1.5 GB collection then ran in 6 minutes with zero errors. The general lesson is that a library's convenience API can have a cost model completely unlike the one its name suggests, and that the cheapest way to tell a network problem from a memory problem is to measure the network separately.

5.2 Throughput of the cleaning pass

Cleaning is regex-bound and single-threaded. Profiling the first version showed that the quality filter cost as much as the entire normalization pipeline, because counting alphanumeric characters with a regex `findall` allocated one Python string per character. Deriving the same ratio from the word list that had already been built, and guarding each markup regex behind a single substring test so documents with no markup skip it, cut the pass time by 45% with byte-identical output.

5.3 Deduplication memory

Exact dedup needs a membership set over 562,070 documents. Storing 64-character hex digests would have cost roughly 130 MB across the two passes on a machine already holding a shuffle buffer and a gzip stream. Truncating to 64-bit integer keys cut that to about 40 MB at a collision probability small enough to be irrelevant next to the near-duplicates exact matching misses anyway.

5.4 Reproducibility

Three sources of nondeterminism had to be closed: dataset contents, sample selection, and the train/val split. Dataset repository commits are pinned in the source registry, the shard visiting order is drawn from a seeded PRNG, and the split uses SHA-256 rather than Python's per-process-salted hash. Each stage writes a manifest recording its inputs, arguments, environment and per-shard SHA-256 digests, so any artifact can be traced back to the run that produced it.

6. Reflections on preprocessing impact

Deduplication matters more than its removal count suggests. Duplicated documents are seen many times by the model at effectively higher learning rate, which encourages verbatim memorization rather than generalization and inflates held-out performance when a duplicate straddles the train/val boundary. The hash-based split used here does not prevent that: a near-duplicate pair can still be split across train and val. Exact dedup before splitting reduces the risk but does not eliminate it.

Lowercasing is the most questionable step in this pipeline. The assignment requires it, and it does reduce vocabulary pressure for word-level models. But GPT-2's byte-level BPE is case-sensitive and was trained on mixed-case text, so lowercasing both destroys information the model could use (proper nouns, acronyms, sentence boundaries) and pushes text off the token distribution the merges were fitted to, which slightly increases tokens per byte. A production pretraining corpus would preserve case and let the tokenizer handle it. This is a clear instance of a preprocessing decision that is defensible in isolation and harmful in combination with a downstream choice.

Quality filtering is a bias decision, not just a cleaning step. The minimum word count alone removed 44,172 documents (7.9% of the input). Those are overwhelmingly Wikipedia stubs, which means the threshold quietly shifted the domain mixture away from encyclopedic text even though the byte budget had been split evenly. Any length or quality threshold is a statement about what the model should consider representative, and reporting per-filter counts per source is the minimum needed to notice when a threshold has moved the distribution.

Packing changes what the model learns about document boundaries. Concatenating documents means most blocks contain a boundary, and without an attention mask that resets at the separator, the model can attend across unrelated documents. The end-of-text token gives it a learnable signal, and at this scale the efficiency gain over padding (44.4% of positions recovered) is worth the cross-document attention. At larger scale, block-diagonal attention masking recovers both.

What would change next. In rough order of expected value: near-duplicate removal via MinHash-LSH; a quality classifier trained to distinguish curated from crawled text rather than the hand-tuned heuristics used here; language identification, since the pipeline assumes English but the sources are not purely English; and preserving case while moving normalization decisions into the tokenizer where they belong.

Appendix: reproducing this corpus

```
pip install -r requirements.txt
python data_collection_preprocessing.py

# or stage by stage
python src/data_collection.py --out-dir data/raw --total-mb 1500
python src/data_cleaning.py --in-dir data/raw --out-dir data/clean
python src/tokenize_corpus.py --in-dir data/clean --out-dir data/tokens --tokenizer gpt2
python src/data_loader.py --tokens-dir data/tokens --sample-out sample_dataset.pt
```

Deliverables: **data_collection_preprocessing.py** plus the four stage scripts in `src/`; **sample_dataset.pt** (10 blocks of 1024 tokens, 162 KB, saved with `torch.save`); and this report. Environment: Python 3.12.3, datasets 5.0.1, seed 20260918.